After spending a week away at my family's yearly beach house rental, I asked myself: what if I could play my PC games from here? What if, better yet, I could control it, with very little latency, from anywhere with decent internet? Fast forward one week, and I had built an entire ecosystem I had never expected: Apollo and Artemis for self-hosted streaming, Tailscale for secure mesh networking, and even an ESP32 that can wake up my sleeping PC remotely. So, yeah, it got interesting. Even a microchip came into play. How did it snowball into this? Allow me to explain:

I had just gotten a brand new Retroid Pocket 5 (RP5), and I was amazed by how I could get to play almost every game I wanted from retro consoles on the go. Nintendo, PlayStation, SEGA, PC… My inner child was feeling very excited because I could experience my childhood games wherever I wanted whenever I wanted, including ones not from portable consoles, like Luigi's Mansion from the GameCube, or PS2 ones. Aside from that, it was an android device, so I could absolutely do anything an android could, which meant Play Store games and apps, or maybe even using the RP5 as a Linux device.

Even though all the above was cool, something came to my mind: what if I wanna play very demanding games? Games that even this device wouldn't run hardware-wise? Well, I knew streaming services were a thing, and they'd gotten way better as time went on, but the input lag for some of them was just not acceptable, namely, shooters. There also was the whole service subscription thing I preferred not to indulge in. There had to be something better. Turns out, these games I could run on my desktop PC no problem. Not only games, but also emulators running very heavy game overhauls. Could I, somehow, play all of them from my RP5, coming from the PC itself?

I knew current software could support streaming and maybe even remote control. Discord, Microsoft Teams, TeamViewer (this one I had worked with a long time ago and my mind exploded as my children eyes contemplated my PC "move on its own"), but none of these were really meant to play games. Okay, so I had to move on from this into something bigger and better. The state of things was simple: I own a moderate to high end PC, a considerably fast internet connection, and my lovely RP5 (I swear I'm not being sponsored). What can I do?

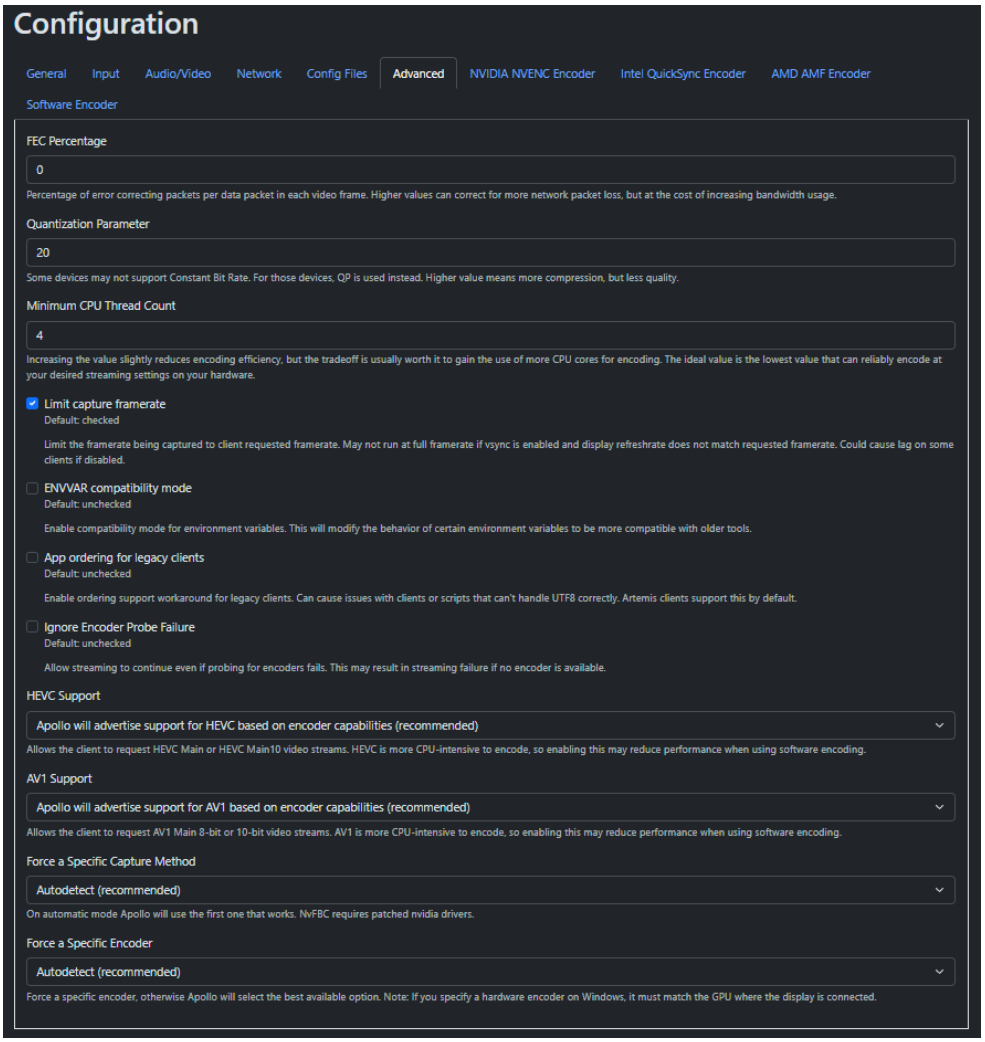
My research began, and after a couple of days, after being deep down the rabbit-hole of host-client streaming software, I found out about Sunshine (see Sunshine for their GitHub) What Sunshine brought to the table was self-hosted game streaming through various configurations that made it really, **really fast.** Seriously, I couldn't believe my eyes once everything was up and running. One of the reasons being it would directly use the GPU's encoder to pack up and send the game capture, which, compared to using the CPU for the same task, meant an immense speedup.

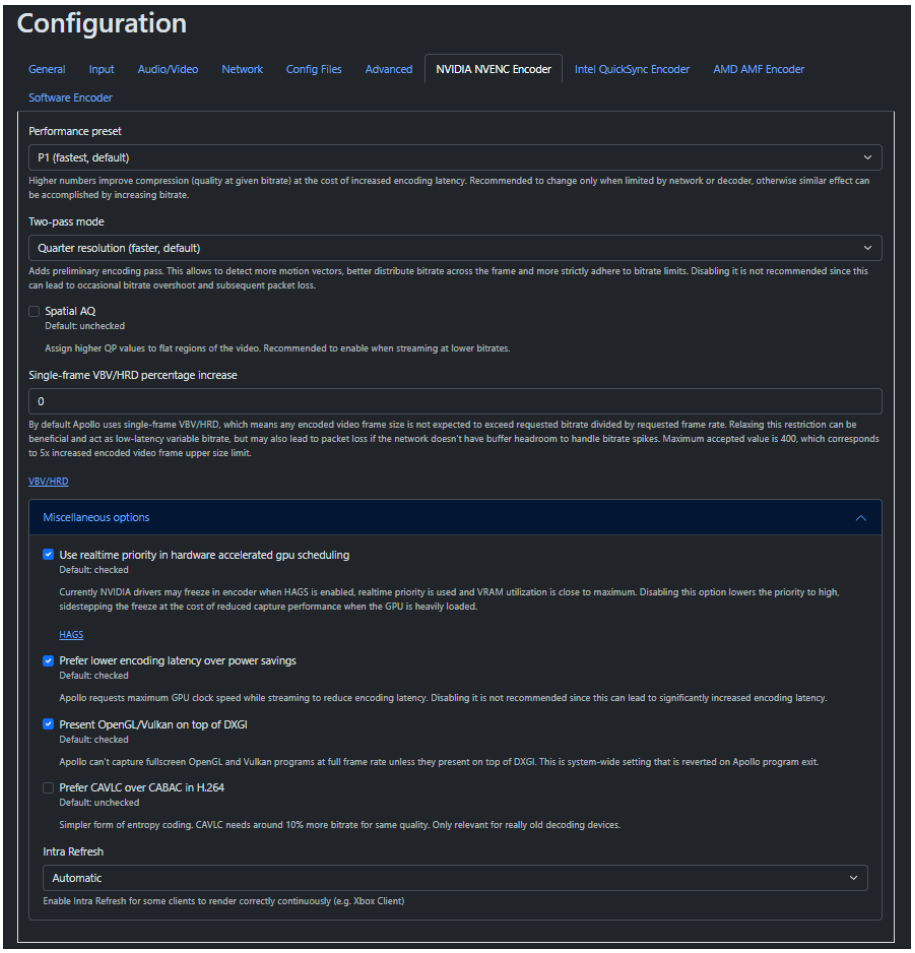
Okay, so that's useful. But how do I connect to it? There must be some sort of client for it, right? Yes, and that's where Moonlight (see Moonlight for their GitHub's android repository) comes into play, a perfect match for Sunshine. What stood out for the Moonlight client was that it uses NVIDIA's GameStream protocol, engineered from the ground up specifically for ultra-low latency gaming (and, in turn, ultra-low latency use cases in general), where mainstream apps often fall short.

Excellent. So, we got everything we wanted: a client and a host. All there was left to do in order to play my desktop games from my RP5 was to install both (Sunshine on the pc, Moonlight on the RP5) and we were good to go. But wait, I said I was deep down the rabbit hole, and it's not too hard to find these if you set the goal, is it? No, not really. And here is where the repo's forks come into play. As a matter of fact, I don't use these two directly. For the client, I actually use a Sunshine fork called Apollo. What this does for me is allow me to stream into the native resolution of the client device by having a built-in virtual display driver (SudoVDA). This is the main distinction, and a big one: what it means is that, once we got everything set up, including the virtual display resolution and refresh rate, every time we connect to our PC, the settings would be preserved and ready for us to enjoy.

Amazing!!! We couldn't ask for more here, except, well, that we could, and for this comes Artemis (originally called Moonlight Noir). Artemis is yet another fork, this time a Moonlight fork, and turns out to be designed by the same guys working under the Apollo project. It's safe to say, then, that using the Apollo-Artemis pair instead brings a lot to the table. Namely, completely automatic virtual display configuration, so no need to tune anything at all, even more latency optimizations, and other quality-of-life stuff, like an improved mouse emulation, shared two-way clipboard (!!!), etc.

So yes, this is the set up I use: Apollo for the server side, and Artemis for the client side. And it works great. Like, seriously. You can't even tell you're streaming. Of course, internet speed is very important here, but it doesn't need that much, and the configuration options are very forgiving. For these I had to dig through some blogs and YouTube videos. Even though it really wasn't necessary because the out-of-the-box settings were good enough, I wanted the absolute best. I didn't even need it, but I wanted to see how good it could get. It was a challenge I wasn't going to drop. I had a mix of different sources and finally arrived at this:


Configuration
General
Input
Audio/Video
Network
Config Files
Advanced
NVIDIA NVENC Encoder
Intel QuickSync Encoder
AMD AMF Encoder
Software Encoder
FEC Percentage
0
Percentage of error correcting packets per data packet in each video frame. Higher values can correct for more network packet loss, but at the cost of increasing bandwidth usage.
Quantization Parameter
20
Some devices may not support Constant Bit Rate. For those devices, QP is used instead. Higher value means more compression, but less quality.
Minimum CPU Thread Count
4
Increasing the value slightly reduces encoding efficiency, but the tradeoff is usually worth it to gain the use of more CPU cores for encoding. The ideal value is the lowest value that can reliably encode at your desired streaming settings on your hardware.
Limit capture framerate
Default: checked
Limit the framerate being captured to client requested framerate. May not run at full framerate if vsync is enabled and display refreshrate does not match requested framerate. Could cause lag on some clients if disabled.
ENVVAR compatibility mode
Default: unchecked
Enable compatibility mode for environment variables. This will modify the behavior of certain environment variables to be more compatible with older tools.
App ordering for legacy clients
Default: unchecked
Enable ordering support workaround for legacy clients. Can cause issues with clients or scripts that can't handle UTF8 correctly. Artemis clients support this by default.
Ignore Encoder Probe Failure
Default: unchecked
Allow streaming to continue even if probing for encoders fails. This may result in streaming failure if no encoder is available.
HEVC Support
Apollo will advertise support for HEVC based on encoder capabilities (recommended)
Allows the client to request HEVC Main or HEVC Main10 video streams. HEVC is more CPU-intensive to encode, so enabling this may reduce performance when using software encoding.
AV1 Support
Apollo will advertise support for AV1 based on encoder capabilities (recommended)
Allows the client to request AV1 Main 8-bit or 10-bit video streams. AV1 is more CPU-intensive to encode, so enabling this may reduce performance when using software encoding.
Force a Specific Capture Method
Autodetect (recommended)
On automatic mode Apollo will use the first one that works. NvFBC requires patched nvidia drivers.
Force a Specific Encoder
Autodetect (recommended)
Force a specific encoder, otherwise Apollo will select the best available option. Note: If you specify a hardware encoder on Windows, it must match the GPU where the display is connected.



Which makes up for pretty much everything that will get you a boost. As for the client (Artemis client for Android), the only stuff you should look out for is video resolution, video frame rate, and video bitrate. For the resolution I noticed it was defaulting to 1080p, which makes a noticeable difference in performance. If your internet and devices allow it, then by all means. But if you are in a less stable network, you might want to turn it down to 720p. Same thing for the video frame rate, crank it down a bit if you're struggling. And lastly, bitrate. This one does make a big difference. Default for me as of right now is set to 20.0 Mbps, but don't be afraid to reduce this a lot if your connection is not fast enough.

Now I could finally play any game I have on my PC or even control my PC from another room. That was great, but I was not having enough with that. I wanted more: what if I could play from anywhere? I was limited by my LAN, and both devices had to be running the services under the same IP range. If that didn't happen, they would just not see each other. I learned about port forwarding when I opened a Minecraft server for my friends in like 2016, but that was not good enough in terms of speed and security (mostly security, I didn't want a port open 24/7). Is there a solution to this issue? Yes, Tailscale.

To my surprise, I didn't know about Tailscale before any of this. Turns out it's quite a big deal, and people/industries use it for its IT, Security, DevOps, and Engineering capabilities. That's fantastic, but what does this mean for us? What it means is that we can have a cluster of our registered devices, both the PC and the RP5 (and, later, the MacBook), and by using identity-based access, Zero Trust networking, and WireGuard encryption, we get to interconnect all of them in an extremely reliable way. They are given what's called private mesh addresses, which are, in this case, IPv4 addresses not reachable from outside our private network, or our "Tailnet". Think of it like an AWS VPC, where your subnet is the Tailnet, and your devices (let's say a couple of EC2's) are our actual devices, the PC and the RP5. This means the devices, once set up, can talk to one another, which means we can stream over the internet!!! This truly is huge news.

After setting everything up and some testing, there it was in all its majesty: an actual way to stream over the internet, where I can use my desktop PC (and play from it) whenever I want, wherever I want, as long as I have an internet connection, and the device I'm using is correctly set up (inside the Tailnet, and having the streaming client). This is good news, too, because Apollo and Artemis are not restricted to a desktop-to-RP5 infrastructure. If you can download them into the devices you need, you can pretty much stream everything to anything. And yes, there are some delay layers, but Apollo and Artemis being as amazing as one can ask, means the delay is pretty good in terms of gaming. You would have to make your adjustments, but you can 100% play stuff using this setup. Shooters…maybe, if you have a very good internet connection both ways, for the client and the server.

There is just one tiny, tiny detail. So tiny, in fact, that I couldn't do anything but laugh when I realized something. What if my PC is in sleep mode? These kinds of services stop when the PC is asleep. It's safe to say that, in my case, I rarely turn off my PC, only when there are updates or every one or two weeks, whatever comes first. I only put it to sleep. For those of you who must without fail turn off the device, you might want to check out the BIOS settings for something called Deep Sleep WoL, or a S5 State, and if it doesn't allow it, then some external hardware device would be needed. For those of you who can keep the device in sleep mode, is this it? Is there no solution to this?

Turns out there is, as always, a solution. Well, several. You could of course ask someone in your house to do you a favor and awake your PC or turn it on, and that would mean you would owe them one, but hey you got your problem fixed. If there is no one, though, what do we do? Turns out there are some requests, called WoL requests (Wake on Lan requests) that devices can send to one another **if they are within the same network** that, when received, the receiving device wakes up. Works like magic. Even the packet sent is called a magic packet. Unfortunately, there isn't any magic involved. It's just a payload with 6 bytes of 0xFF followed by the target device's MAC address, repeated 16 times. For a sample MAC address of XY:XY:XY:XY:XY:XY, it would be something like

FFFFFFFFFFFF XYXYXYXYXYXY XYXYXYXYXYXY … (16 times)

And that fixes the remote waking up to some extent, but not entirely. That will work if you're inside the same network, but isn't the whole point not being in the same network? Could we use this WoL request remotely?

Well, I worked with what I got, and that is an ESP32. I got an ESP32 back in high school (2022 term) for a subject called Intelligent Systems, and I knew right away it was full of potential. Having integrated Wi-Fi and Bluetooth made it a bargain for the price, among other things. And this was no exception: **by registering our ESP32 as a node inside our Tailnet, embedding code into the ESP32 that sends magic packets to some other device's address, and connecting it to the same local network, we could bypass our LAN limitations and talk to the ESP32 through any other node inside the Tailnet, making it send a WoL request locally to the device we want to wake up.** We can load executables and code of a certain size into our ESP32, which autoruns everything that can be run once it receives power.

As for the code, the message itself isn't hard to implement because, like we saw earlier, a magic packet is very simple. What needs some engineering is getting our chip to automatically connect to our Tailnet every time, while preserving a listening state for us that can "relay" our message whenever we want. Luckily for us, someone else has been in this position, (somehow). All we need to do is follow the README instructions, which are straightforward. The only thing the author doesn't tell you is how to get this Tailscale auth key the configuration file is asking for, but it's a fairly easy process. You just have to log in into your Tailscale admin console, go to Settings, then Keys, and then you click on the "Generate auth key" button. Some settings will show for your key, and you choose the ones that best fit your needs.

After generating the key and pasting it into the configuration file and running the necessary commands through the idf.py interface, the device is ready. Now, by having it in a flashed state, the code will run whenever the chip gets power. Now it's just a matter of keeping the device always running, which is easy to do, as it needs a mere 3.3V input. Now, whenever you want to wake up your PC remotely, you just have to send any UDP message to the ESP32 through the Tailscale network on port 9999 (default one being used inside the code), which we do with our RP5. Tailscale gives us a mesh IP for our ESP, which we can insert into any Wake on Lan app there is on the Google Play Store, or manually send the packet with a tool like netcat by specifying the right parameters (for the message, it doesn't matter; it can be anything). Once the listener picks up the request, it sends the magic packet to our pc locally, which makes it fire up our Apollo and Tailscale services, which allows us to connect to it using, not the local IP address now, but the one Tailscale gives us. If you're connecting locally, then you may use either, but

of course keeping it local will make the delay almost nonexistent, so always prefer just that.

That’s it. After I got this working I installed the Artemis client into my MacBook, and since everything was already dealt with, I could just use my PC remotely. All of this couldn’t have been possible had my family not rented a beach house for 1 month which made me think “hey, it would be cool if I could play my PC games from here”. And there you have it: your games anywhere you want (technically). The knowledge gained can be extrapolated into a whole bunch of other use cases, and the network concepts are very valuable, too, which is why I wanted to write this blog. I had a fun time getting to know all these pieces of software and hardware, and how they interacted with one another.